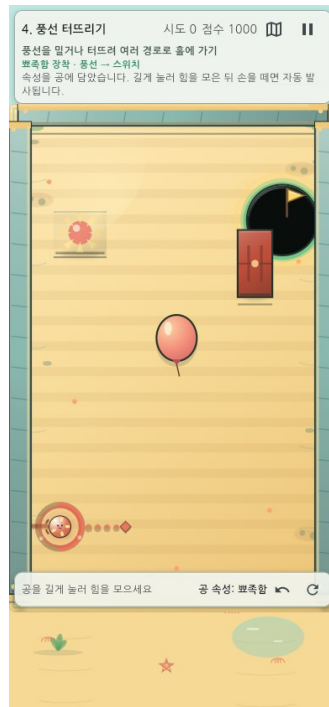


GAME · AI · PRODUCT CASE STUDY

속성 한방(Property Shot)

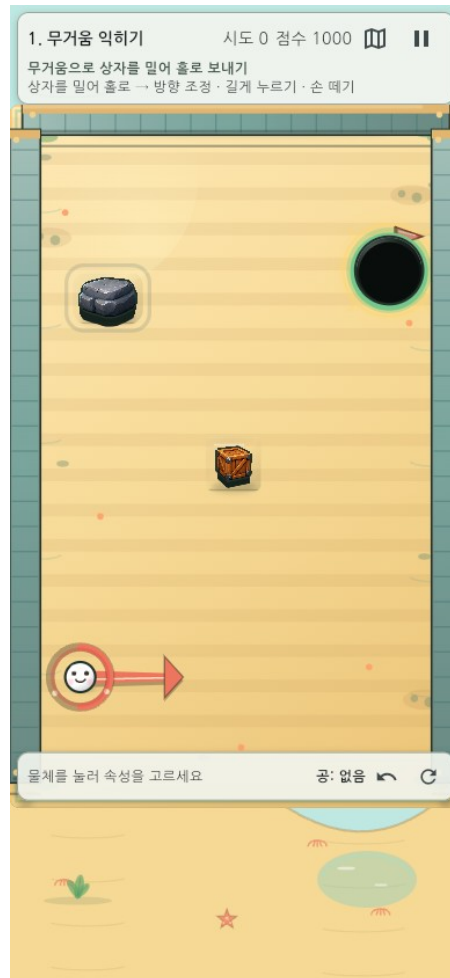


2026-08-10 KST

NAN 2026 Game × AI 해커톤 사전 과제

1. 프로젝트 요약

속성 한방은 환경 오브젝트의 물리 속성을 공에 옮기고, 바뀐 장면과 잔류 공을 다음 해법으로 사용하는 세로형 물리 퍼즐이다. 10단계 × 4패턴의 플레이 가능한 수직 프로토타입을 기획·구현·검증하고, AI 에이전트를 작은 제작 조직처럼 운영했다.



속성 한방의 실제 모바일 플레이 화면

항목	내용
역할	게임 디렉팅, 시스템 설계, AI 오케스트레이션, 품질 기준 결정
플랫폼	Flutter / Flame · Web / Android
범위	10단계·40패턴, 난이도, 힌트, 보상, 리플레이, 오늘의 도전
핵심 성과	상태가 누적되는 물리 퍼즐, 검증 가능한 AI 협업, 완성된 Web·Android 빌드

2. 해결하려 한 문제

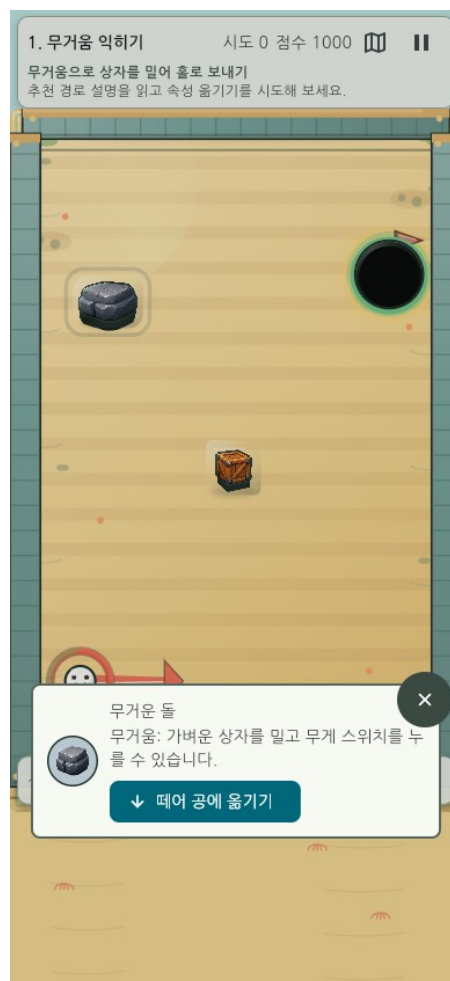
조준 퍼즐은 방향과 힘만 맞추면 반복적으로 느껴지기 쉽다. 반대로 기믹이 많아지면 초보자는 무엇을 해야 하는지 모른다. 이 프로젝트는 두 문제를 함께 풀었다.

- 속성을 옮겨 장면 자체를 바꾸므로 매 샷의 의미를 늘린다.
- 실패한 공과 기물 상태가 남아 시행착오가 정보와 자원이 된다.
- 쉬움 모드·구체적 L1/L2 힌트·선택형 열쇠로 정답 공개 없이 진입 장벽을 낮춘다.

3. 핵심 경험 설계

3.1 플레이 루프

관찰 → 속성 선택 → 방향·힘 결정 → 물리 연쇄 → 바뀐 상태 재사용의 순환을 만들었다.



속성 원본에서 공으로 능력을 옮기는 선택 화면

3.2 실패를 자원으로 바꾸기

첫 샷이 홀에 들어가지 않아도 공이 남고 문·스위치·반사판 상태가 유지된다. 플레이어는 실패를 초기화하지 않고 다음 샷의 발판으로 바꾼다. 이는 속성 한방의 가장 중요한 차별점이다.

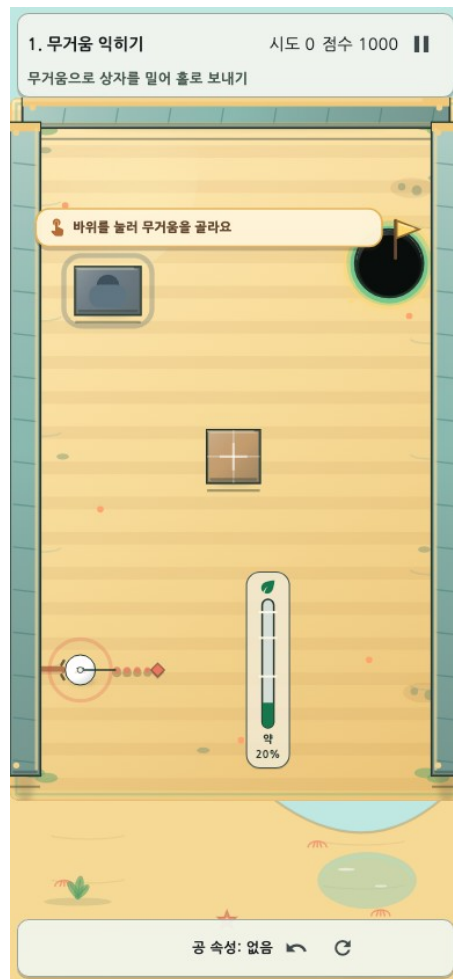
3.3 10단계 학습 설계

무거움과 탄성에서 시작해 스위치·풍선·비워진 원본·파워 발판·잔류 공·회전 반사판·속성 통합으로 확장했다. 단계별 4개 변형은 같은 규칙을 다른 공간에서 다시 적용하게 한다.

4. 대표 UX 개선 사례

4.1 손가락에 가리는 게이지

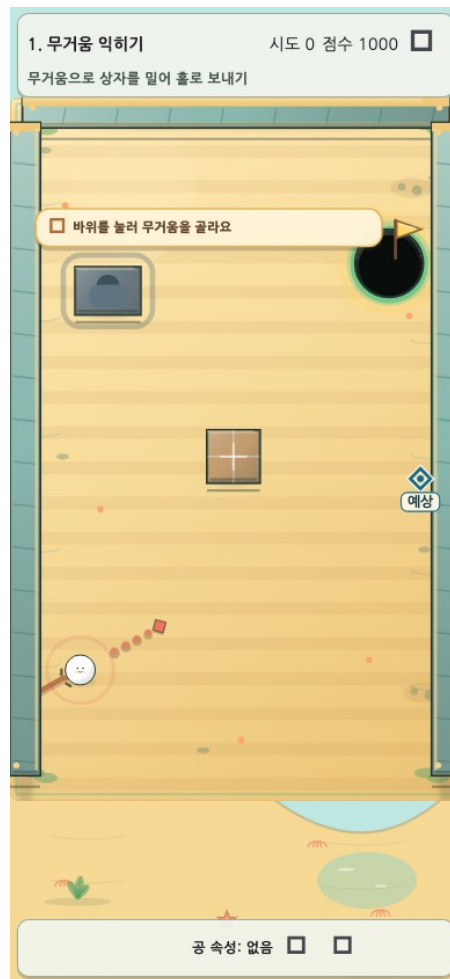
구분	내용
문제	공 주변 원형 게이지가 모바일에서 손가락에 가렸다.
판단	발사 감각과 입력 임계값은 바꾸지 않고 표시 위치만 분리했다.
결과	공 가까이의 부유 직선 rail이 핵심 기물과 열쇠를 피하며, 좌·우 손 선호와 저모션을 지원한다.



모바일 화면에서 기물을 피하는 부유 충전 게이지

4.2 너무 어려운 첫 경험

구분	내용
문제	첫 플레이에서 복잡한 변형이 무작위로 나올 수 있었다.
판단	학습 구간만 완화하되 물리와 보통 기록의 공정성은 유지했다.
결과	첫 1-4단계는 기준 패턴을 우선하고, 쉬움 모드는 실제 판정 결과의 첫 도착점만 표시한다.



쉬움 모드 예상 도착 표시

4.3 추상적인 도움말

구분	내용
문제	‘기막을 활용하세요’는 실제 행동을 안내하지 못했다.
판단	정답 숫자를 바로 공개하지 않고 행동의 구체성을 단계별로 높였다.
결과	40패턴의 L1은 첫 행동을, L2는 위치·세기·순서·결과를 안내하며, 보상 또는 열쇠를 선택한 사람에게만 열린다.

현재 스테이지 팁 · 1단계

오른쪽 아래 젤리의 탄성을 공에 옮긴 뒤, 홀을 바로 겨냥하지 말고 바닥 반사를 먼저 이용해 보세요.

다음 단계는 정확한 각도나 힘 대신, 다음에 시험할 기믹과 순서를 알려줍니다.

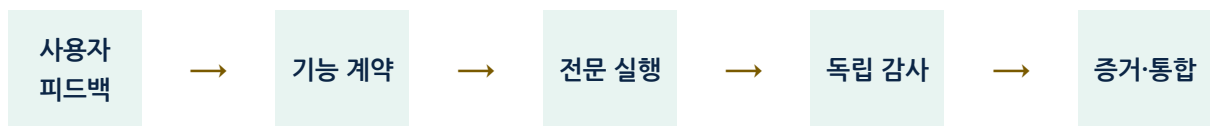
 더 구체적으로

선택한 패턴에 맞춰 첫 행동을 알려 주는 L1 힌트

5. AI 디렉팅 방식

AI를 한 명의 만능 개발자로 쓰지 않고 기능 담당·레벨 담당·문서 담당·독립 검사로 나눴다.

프롬프트는 목적, 보호해야 할 규칙, 테스트 기준, 종료 조건을 포함했다.



감사에서 문제가 발견되면 기능 계약 단계로 돌아간다.

단계	내 결정	AI의 기여
문제 정의	어떤 플레이 경험을 바꿀지 결정	코드·화면·데이터에서 원인 탐색

단계	내 결정	AI의 기여
설계	공정성·접근성·기록 정책 확정	대안과 영향 범위 비교
구현	우선순위와 공유 파일 소유권 관리	기능·테스트·문서 구현
검증	통과 기준과 반려 여부 결정	독립 감사, 회귀·시각·결정론 확인

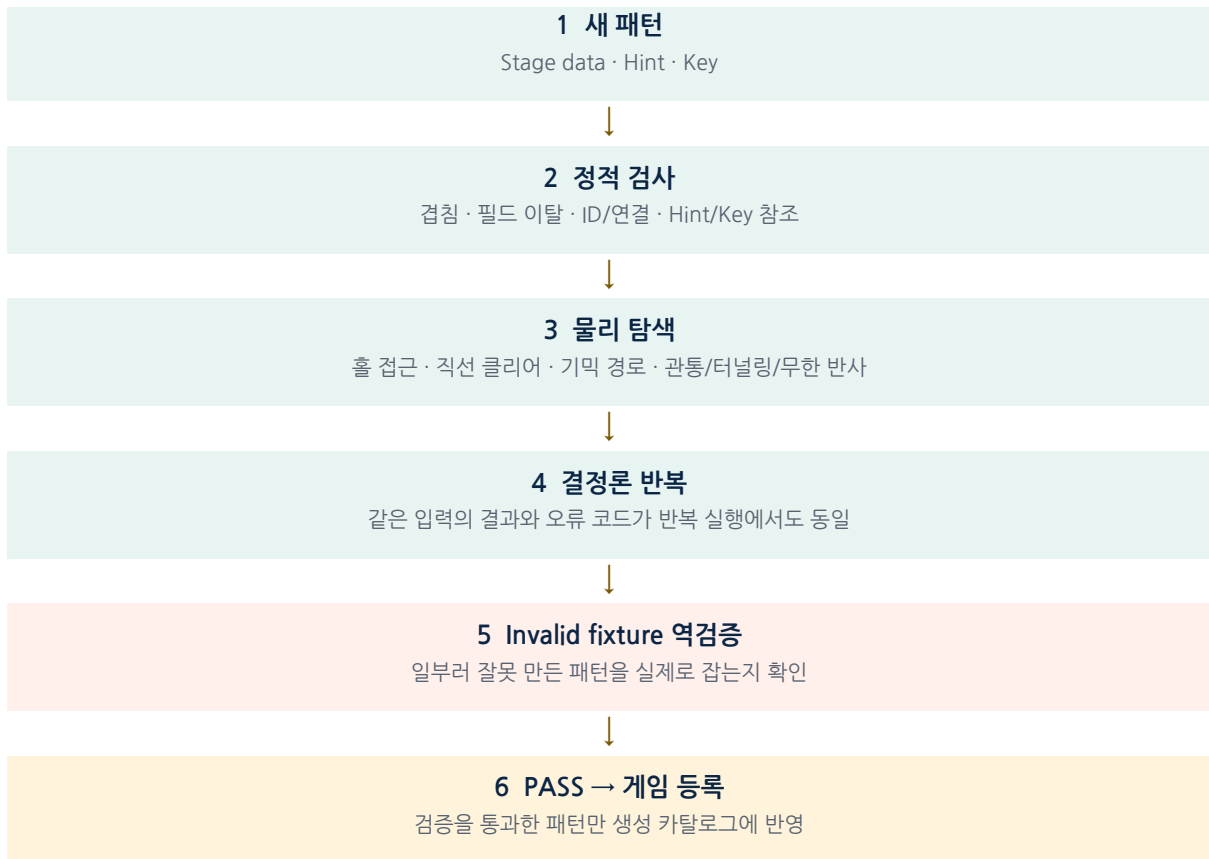
가장 중요한 학습은 AI가 만든 검증 지표도 틀릴 수 있다는 점이었다. 레벨 우위 계산에서 서로 다른 표본 수를 나눈 오류를 감사 에이전트가 찾아냈고, 동일 입력 격자 비교로 계약을 다시 만들었다.

6. 기술 구조와 검증 체계

계층	설계 의도
데이터 기반 StageCatalog	40패턴을 코드 수정 없이 검증·생성
결정론 ShotResolver	리플레이와 미리보기, 테스트가 같은 판정 사용
RunState v3	잔류 공·보상·힌트·열쇠를 안전하게 복구
Flutter + Flame	적응형 UI와 Canvas 게임판 분리
Validator + Golden	구조·물리·접근성·화면을 자동 검사

6.1 AI가 만든 레벨을 바로 등록하지 않았다

AI가 만든 패턴은 문법상 정상이어도 홀로 가는 직선이 열려 있거나, 기물이 겹치거나, 특정 입력에서 무한 반사가 날 수 있다. 그래서 생성과 등록 사이에 StagePatternValidator를 두었다.



검증기가 있다고 믿지 않고, 검증기가 오류를 잡는지도 다시 검증했다.

정적 구조와 실제 물리를 통과한 뒤에도 같은 입력의 결과가 반복 실행에서 같아야 한다.
마지막으로 겹침·잘못된 참조·홀 관통·무한 반사를 일부러 넣은 invalid fixture를 사용해, 검증
도구가 오류를 실제로 잡는지도 확인했다. 이 구조는 AI가 만든 결과를 자동 채택하지 않고 **검증
가능한 후보**로 다룬다는 제작 원칙을 코드로 만든 사례다.

7. 결과와 품질 증거

- 10단계·40패턴 모두 플레이 가능하며 각 단계에서 기믹 사용이 우회보다 유리하다.
- 320×568 모바일부터 태블릿까지 보드·보상·게이지·SafeArea 화면을 검증했다.
- 난이도 변경·앱 재개·크래시 복구에서도 보통 기록이 오염되지 않는다.
- Web 공개 빌드와 Android release 빌드를 생성했다.



모바일 클리어 보상 화면

8. 내가 만든 가치

1. 단순 조준 게임을 '장면 상태를 편집하는 퍼즐'로 확장했다.
2. 초보 지원과 기록 공정성을 동시에 만족하는 난이도 정책을 만들었다.
3. 자연어 피드백을 데이터와 테스트 증거로 전환하는 AI 제작 체계를 설계했다.
4. 잘못된 AI 결과를 독립 감사로 반려하고 수정하는 운영 문화를 만들었다.
5. 게임, 기술 문서와 편집 가능한 발표 자료를 하나의 결과물로 정리했다.

9. 포트폴리오 및 참고자료

본문은 플레이 경험과 주요 판단만 다룬다. 아래 자료는 설계 과정이나 검증 근거를 더 확인할 때 선택적으로 볼 수 있는 원문이다.

참고자료	확인할 수 있는 내용
프롬프트 및 결과 기록	자연어 요구가 작업 계약과 결과로 바뀐 과정
StagePatternValidator 검증	패턴 구조·물리·invalid fixture의 검증 방식
기믹 우위 검증	준비 전·후를 같은 입력 격자에서 비교한 방법
힌트 시스템 설계	L1/L2·열쇠·보상·저장 복구의 연결
접근성 검증	화면 읽기·저모션·큰 글자·SafeArea 대응
에셋 권리대장	생성형 AI·외부 에셋·오픈소스의 사용 범위